

STAGE D'IMMERSION EN ENTREPRISE

**Sujet: Automatisation intelligente de
la plateforme e-commerce Prigado**

Réalisé par: Sofiene Zayati

Entreprise: Inspark Connect

2025/2026

Remerciements

Je tiens à exprimer ma sincère gratitude à l'équipe d'Inspark Connect pour l'accueil, la confiance et l'accompagnement tout au long de mon stage. Je remercie particulièrement Mme Salma Sayah pour son encadrement et ses conseils.

Je remercie également les enseignants et l'administration d'ESPRIT pour la formation qui m'a permis de mener ce projet technique dans un contexte professionnel réel.

Liste des abréviations

Abréviation	Définition
IA	Intelligence artificielle
API	Application Programming Interface (interface de programmation applicative)
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
JSON	JavaScript Object Notation
BDD	Base de données
SEO	Search Engine Optimization

Table des matières

Remerciements	i
Liste des abréviations	ii
Introduction	1
1 Présentation du projet	2
1.1 Introduction	2
1.2 Description du projet	2
1.3 Vue d'ensemble architecturale	3
1.4 Étude de l'existant	4
1.5 Solution proposée	4
1.6 Choix méthodologiques	4
1.6.1 Approche itérative	4
1.6.2 Démarche d'implémentation	4
1.7 Conclusion	4
2 Analyse et spécification des besoins	5
2.1 Introduction	5
2.2 Identification des acteurs	5
2.3 Spécification des besoins fonctionnels	5
2.4 Spécification des besoins non fonctionnels	5
2.5 Diagramme des cas d'utilisation générale	6
2.6 Conclusion	6
3 Conception	7
3.1 Introduction	7
3.2 Patron d'architecture commun (AI Router Model)	7
3.3 Conception des prompts IA	8
3.3.1 Contraintes de structure et extraction d'intention	8
3.3.2 Guard clauses et sorties robustes	8
3.3.3 Sorties structurées orientées métier	9
3.4 Compétences techniques mobilisées	9

4	Réalisation	10
4.1	Introduction	10
4.2	Framework et technologies utilisés	10
4.3	Environnement de travail	10
4.4	Module 1 – Assistant de création de fiches produits (EPIC 4)	10
4.4.1	Problématique métier	10
4.4.2	Solution proposée	11
4.4.3	Le cerveau du système : architecture multi-parcours A/B/C	11
4.4.4	Fonctionnalités principales	12
4.4.5	Extraction et écriture des données (BDD) – Module Product Creation	12
4.4.6	Illustrations de la démonstration (présentation)	13
4.5	Module 2 – Prigado Boost : assistant ventes et marketing (EPIC 5)	17
4.5.1	Intentions implémentées	17
4.5.2	Fonctions réalisées	17
4.5.3	Extraction et transfert des données (BDD) – Module Boost	18
4.5.4	Illustrations de test du module Boost (chat + sorties)	19
4.6	Module 3 – Prigado Logistics : recommandation logistique et commerciale (EPIC 6)	23
4.6.1	Intentions implémentées	23
4.6.2	Fonctions réalisées	23
4.6.3	Extraction et algorithmes (BDD) – Module Logistics	24
4.6.4	Illustrations de test du module Logistics (chat + sorties)	25
5	Défis techniques et solutions apportées	27
5.1	Défi 1 – Gestion de l’absence de données	27
5.1.1	Problème	27
5.1.2	Solutions mises en œuvre	27
5.2	Défi 2 – Agrégation des résultats SQL pour l’IA	27
5.2.1	Problème	27
5.2.2	Solution	28
5.3	Défi 3 – Consistance des données entre branches	28
5.3.1	Problème	28
5.3.2	Solution	28
5.4	Défi 4 – Fiabilité des sorties IA	28
5.4.1	Problème	28
5.4.2	Solution	28
6	Intégration avec la base de données	29
6.1	Tables mobilisées	29
6.2	Complexité relationnelle et choix de modélisation	29
6.2.1	Relations exploitées dans les workflows	29

6.2.2	Pourquoi la relation était complexe en pratique	30
6.2.3	Colonnes clés mobilisées pour obtenir les meilleurs résultats	30
6.2.4	Impact direct sur la qualité des sorties IA	30
6.3	Exemple de requêtes SQL utilisées	31
Conclusion générale et perspectives		33
Bibliographie		35

Table des figures

1.1	Architecture globale du projet Prigado	3
2.1	Cas d'utilisation généraux du système IA Prigado	6
3.1	Architecture globale des assistants IA Prigado	8
4.1	Système multi-parcours piloté par AI Router	12
4.2	Workflow de création assistée des fiches produits	13
4.3	Module 1 - Path A (prix fourni et prix manquant)	14
4.4	Module 1 - Génération de tags produit	15
4.5	Module 1 - Path B (produit unitaire et liste complète)	16
4.6	Module 1 - Path C (generalHelp)	17
4.7	Workflow Prigado Boost (ventes et marketing)	19
4.8	Module 2 - Test launchCampaign (chat + email)	20
4.9	Module 2 - Test getSalesReport (Sofiene et Razer)	21
4.10	Module 2 - Test suggestPromo	21
4.11	Module 2 - Test createSocialPost (topic spécifique)	22
4.12	Module 2 - Test createSocialPost (topic normalisé top-selling)	23
4.13	Workflow Prigado Logistics (analyse prédictive et risque)	25
4.14	Module 3 - Test predictActivityPeak	25
4.15	Module 3 - Test recommendTrendingProducts (avec fallback no-data)	25
4.16	Module 3 - Test checkCustomerRisk (email et nom client)	26

Introduction

Dans le cadre de mon stage de deux mois chez Inspark Connect, j'ai participé au projet Prigado, une plateforme e-commerce basée sur Vue.js et Laravel. L'objectif principal de ma mission était d'intégrer une couche d'intelligence artificielle opérationnelle via n8n afin d'automatiser des processus métier (marketing, analyse commerciale, support décisionnel logistique et création assistée de fiches produits). n8n est une plateforme open-source d'orchestration de workflows permettant d'automatiser des processus et d'intégrer différents services via une interface visuelle basée sur des nœuds.

La démarche adoptée ne consistait pas à ajouter une IA isolée, mais à concevoir des workflows robustes reliés aux données réelles de la plateforme (base MySQL), capables de comprendre des requêtes en langage naturel et de produire des actions concrètes.

Problématique Malgré la présence d'une plateforme e-commerce fonctionnelle, plusieurs opérations restaient manuelles et chronophages : création de fiches produits, analyse commerciale, préparation de campagnes marketing et évaluation du risque client.

L'objectif de ce stage était donc de concevoir et intégrer des assistants basés sur l'intelligence artificielle capables d'automatiser ces processus tout en restant connectés aux données réelles de la plateforme.

Chapitre 1

Présentation du projet

1.1 Introduction

Ce chapitre présente le cadre général du stage, le contexte métier de Prigado et les choix retenus pour la mise en place des assistants IA.

1.2 Description du projet

Inspark Connect est une entreprise orientée transformation digitale et développement de solutions logicielles. Dans ce contexte, la plateforme Prigado représente un cas d'usage e-commerce complet, avec des enjeux de performance commerciale, d'expérience utilisateur et d'automatisation des opérations.

1.3 Vue d'ensemble architecturale

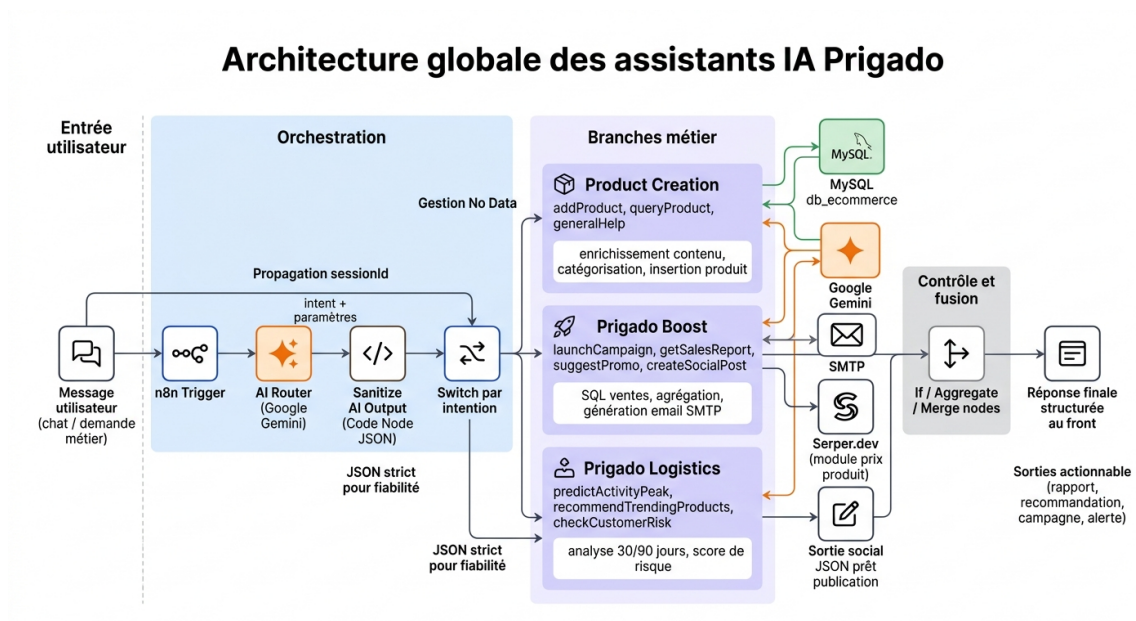


FIGURE 1.1 – Architecture globale du projet Prigado

Prigado est une marketplace e-commerce avec :

- un front-end Vue.js,
- un back-end Laravel,
- une base de données MySQL (produits, utilisateurs, commandes, détails de commandes, catégories, etc.).

L'enjeu du stage était d'ajouter des assistants IA opérationnels s'appuyant sur :

- n8n pour l'orchestration,
- Google Gemini pour l'analyse et la génération,
- MySQL pour l'accès aux données métier,
- SMTP pour l'envoi de campagnes email,
- (module création produit) Serper.dev pour la recherche de prix de marché,
- (module social) génération de contenu structuré prêt à publier sur réseaux sociaux.

Sur la base des exports n8n actuellement disponibles, la branche sociale produit un JSON de publication (texte + idée visuelle), sans nœud explicite de publication API (HTTP Request / Facebook Graph API) dans le workflow exporté.

L'approche mise en place suit un principe commun sur tous les modules : partir d'un besoin métier exprimé en langage naturel, le transformer en intention exploitable, puis exécuter une chaîne d'actions data+IA mesurable et reproductible.

1.4 Étude de l'existant

Avant l'intégration IA, plusieurs opérations restaient majoritairement manuelles : rédaction des fiches produits, extraction des indicateurs de vente, préparation des campagnes et analyse du risque client. Ces tâches étaient réalisables, mais coûteuses en temps et sensibles aux erreurs de formatage et d'interprétation.

1.5 Solution proposée

La solution retenue repose sur des workflows n8n pilotés par un AI Router, connectés à Google Gemini et à la base MySQL. Cette approche permet de transformer une demande en langage naturel en action métier exécutable : insertion produit, rapport de ventes, recommandation marketing, prédiction logistique et scoring de risque.

1.6 Choix méthodologiques

1.6.1 Approche itérative

Le développement a suivi une logique itérative : conception d'un premier flux minimal, tests sur cas réels, correction des cas limites, puis enrichissement progressif des branches métier.

1.6.2 Démarche d'implémentation

La démarche a combiné trois axes : fiabilité des sorties IA (JSON structuré), robustesse des flux n8n (Switch/If/Aggregate/Merge) et qualité des données SQL (jointures et filtrage métier).

1.7 Conclusion

Cette étape de cadrage a permis de définir une architecture cible claire et une démarche de réalisation progressive, directement alignée sur les besoins opérationnels de Prigado.

Chapitre 2

Analyse et spécification des besoins

2.1 Introduction

Ce chapitre formalise les besoins du système afin de relier explicitement les cas d’usage métier aux modules techniques implémentés.

2.2 Identification des acteurs

- **Administrateur / équipe métier** : pilote les campagnes, consulte les rapports, suit les tendances.
- **Vendeur** : suit ses performances commerciales et ses produits.
- **Client** : passe des commandes ; son historique alimente l’analyse de risque.
- **Équipe technique** : configure les workflows n8n et contrôle la qualité des intégrations IA/BDD.

2.3 Spécification des besoins fonctionnels

- Ajouter et enrichir automatiquement une fiche produit à partir d’une demande conversationnelle.
- Récupérer des informations produit via des requêtes en langage naturel.
- Lancer des campagnes email personnalisées.
- Générer des rapports de ventes par vendeur et proposer des bundles promotionnels.
- Identifier des produits tendance et prédire les pics d’activité.
- Évaluer automatiquement le risque client à partir de l’historique de commandes.

2.4 Spécification des besoins non fonctionnels

- **Fiabilité** : garantir une sortie exploitable même en cas de données vides.

- **Maintenabilité** : structurer les workflows en branches claires et testables.
- **Performance** : limiter les temps de réponse pour un usage conversationnel fluide.
- **Sécurité** : contrôler l'accès aux données métier et éviter les sorties non valides.
- **Traçabilité** : conserver un contexte d'exécution (session) et des étapes de transformation explicites.

2.5 Diagramme des cas d'utilisation générale

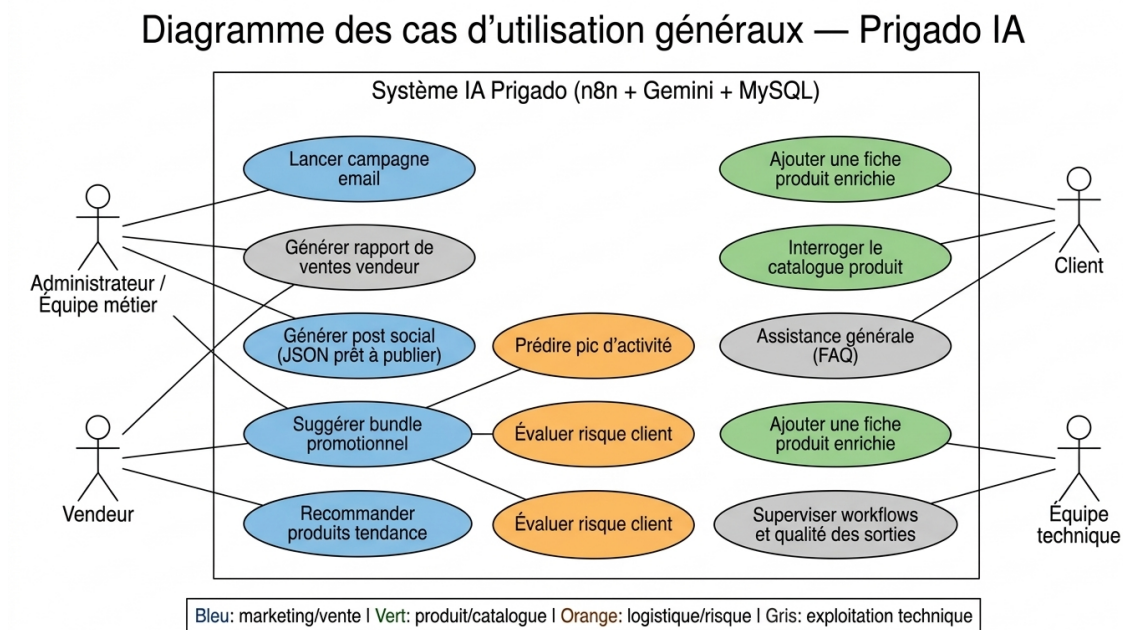


FIGURE 2.1 – Cas d'utilisation généraux du système IA Prigado

2.6 Conclusion

Les besoins identifiés ont servi de base pour concevoir une architecture modulaire orientée intents, permettant de couvrir des usages métier différents avec un socle technique commun.

Chapitre 3

Conception

3.1 Introduction

Cette phase de conception détaille le patron d'architecture commun et les décisions techniques ayant permis de rendre les workflows évolutifs et robustes.

3.2 Patron d'architecture commun (AI Router Model)

Les workflows réalisés suivent une architecture modulaire et évolutive :

1. **Trigger conversationnel** : réception d'un message utilisateur.
2. **AI Router** : interprétation de la requête en langage naturel et extraction d'un `intent`.
3. **Sanitization JSON** : normalisation de la sortie IA via un `Code Node` pour garantir un JSON exploitable.
4. **Conservation du contexte** : propagation du `sessionId` et intégration d'un nœud **Simple Memory (Buffer Window Memory)** LangChain qui permet à l'assistant IA de conserver un historique conversationnel fluide.
5. **Switch par intention** : routage vers la branche métier appropriée.
6. **Exécution métier** : requêtes MySQL, logique conditionnelle, agrégation.
7. **Agent IA spécialisé** : reformulation métier (rapport, recommandation, email, etc.).
8. **Merge final** : convergence de toutes les branches vers une sortie unique.

Exemple de sortie attendue après routage IA :

Listing 3.1 – Exemple d'objet JSON routé par l'AI Router

```
{
```

```

"intent": "getSalesReport",
"seller_name": "Razer",
"limit": 3,
"sessionId": "..."
}

```

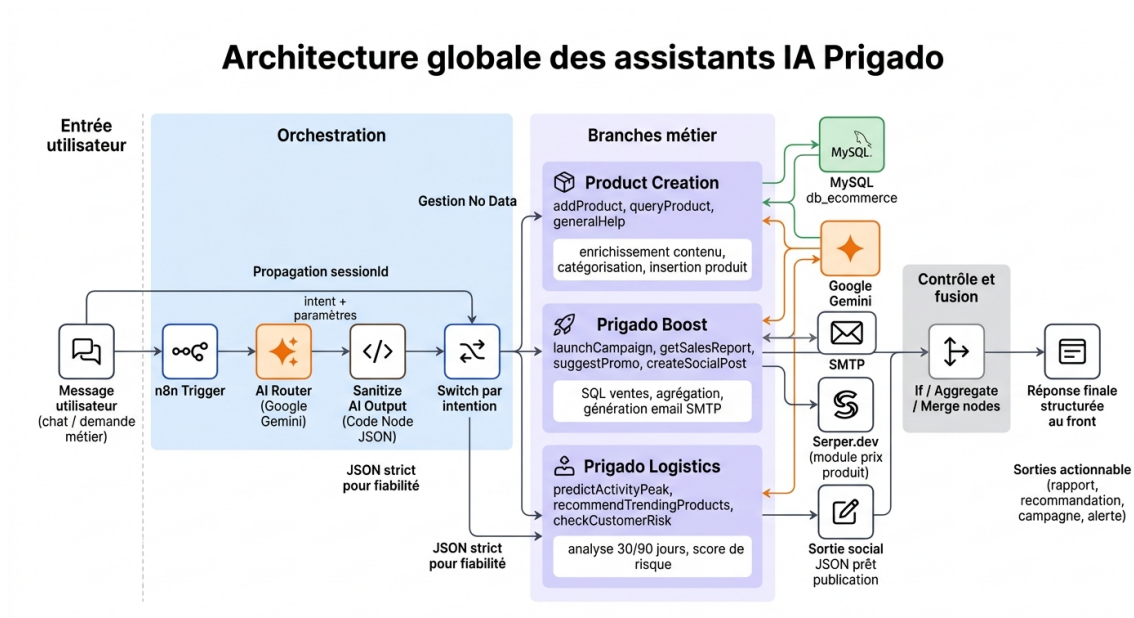


FIGURE 3.1 – Architecture globale des assistants IA Prigado

3.3 Conception des prompts IA

La qualité des sorties dépendait directement de la conception des prompts, notamment pour fiabiliser le passage entre langage naturel, SQL et réponse métier exploitable.

3.3.1 Contraintes de structure et extraction d'intention

Les prompts ont imposé une sortie structurée pour le routage :

- extraction explicite de l'intent,
- format JSON strict (sans texte hors objet),
- champs minimaux obligatoires selon la branche,
- normalisation des noms de variables avant exécution.

3.3.2 Guard clauses et sorties robustes

Pour éviter les réponses ambiguës, les prompts intégraient des règles de garde :

- message explicite en cas d'absence de données,

- distinction entre "entité non trouvée" et "entité trouvée sans résultats",
- sorties reformulées pour usage métier direct (rapport, recommandation, alerte).

3.3.3 Sorties structurées orientées métier

Les prompts ont été conçus pour produire des sorties directement intégrables dans le flux :

- rapports synthèses lisibles pour les décideurs,
- contenus marketing prêts à l'emploi,
- recommandations logistiques actionnables,
- objets JSON compatibles avec les étapes de **Switch/Merge**.

3.4 Compétences techniques mobilisées

- **Prompt Engineering** : définition de consignes strictes, formats JSON imposés, gestion des cas limites.
- **Architecture de workflows** : conception de systèmes non linéaires avec **Switch**, **If**, **Aggregate**, **Merge**.
- **Intégration systèmes** : connexion de l'IA à la base MySQL et aux services externes.
- **Vision produit/métier** : traduction d'un besoin business en logique technique exécutable.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la mise en œuvre pratique des workflows IA, les technologies mobilisées et les résultats obtenus sur les trois modules fonctionnels.

4.2 Framework et technologies utilisés

- **n8n** : orchestration des workflows et intégration des services.
- **Google Gemini** : interprétation des intentions et génération de sorties métier.
- **MySQL** : stockage et interrogation des données e-commerce.
- **Laravel** : socle back-end de la plateforme Prigado.
- **Vue.js** : interface front-end de la plateforme.
- **SMTP** : envoi automatisé de campagnes email.

4.3 Environnement de travail

Le travail a été réalisé dans un environnement de développement intégré, avec export et validation des workflows n8n, tests incrémentaux sur des jeux de données réelles et vérification des sorties IA avant intégration finale.

4.4 Module 1 – Assistant de création de fiches produits (Epic 4)

4.4.1 Problématique métier

Avant l'intégration de l'assistant, la gestion du catalogue produit faisait face à plusieurs difficultés opérationnelles :

- **Saisie manuelle lente** : ajout des produits champ par champ, avec un risque élevé d'erreurs.
- **Données hétérogènes** : descriptions, tags et catégories non homogènes selon les utilisateurs.
- **Recherche chronophage** : vérification manuelle des prix et rédaction des descriptions via navigation web.
- **Accès difficile à l'information** : des questions simples sur le catalogue nécessitaient un accès technique (BDD/développeur).

Ces constats ont servi de point de départ pour une approche orientée assistant conversationnel.

4.4.2 Solution proposée

La solution retenue est un assistant en langage naturel capable de :

- ajouter un produit avec enrichissement automatique,
- répondre à des questions sur le catalogue via la base de données,
- traiter des questions générales de type chatbot (politiques magasin, FAQ).

Cette logique est implémentée via un système multi-parcours A/B/C piloté par l'AI Router.

4.4.3 Le cerveau du système : architecture multi-parcours A/B/C

Le diagramme fonctionnel utilisé dans la présentation suit la logique suivante :

- **Entrée** : message utilisateur.
- **AI Router** : identification de l'intention principale.
- **Path A – Add Product** : enrichissement + insertion.
- **Path B – Get Product Info** : requête BDD + formatage.
- **Path C – General Question** : réponse chatbot basée sur une base de connaissances.

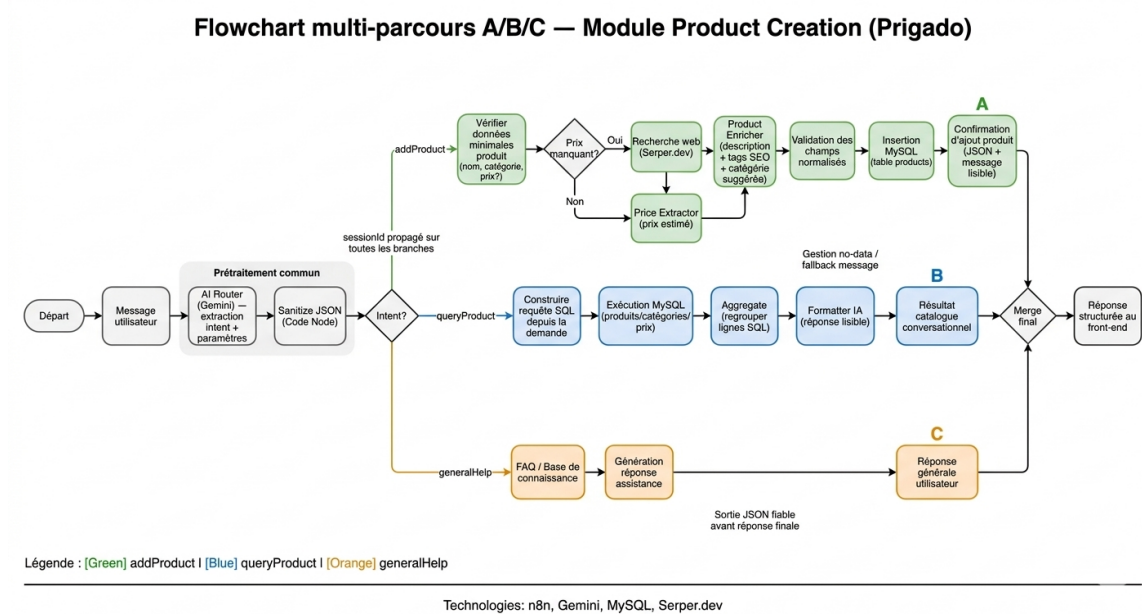


FIGURE 4.1 – Système multi-parcours piloté par AI Router

4.4.4 Fonctionnalités principales

- **Analyse d'intention** : addProduct, queryProduct, generalHelp.
- **Prix manquant** : appel API externe (Serper.dev) puis extraction d'un prix pertinent via un agent de type « Price Extractor ».
- **Enrichissement de contenu** : génération de description commerciale et de tags SEO.
- **Catégorisation assistée** : proposition de `category_id` à partir des catégories disponibles, via un agent de type « Product Enricher ».
- **Insertion MySQL** : enregistrement final du produit enrichi dans la table `products`.
- **Confirmation utilisateur** : un agent IA final génère un message de succès formaté reprenant les détails du produit nouvellement inséré.
- **Requêtes conversationnelles** (queryProduct) : agrégation des lignes SQL en tableau JSON unique via `Aggregate` puis reformulation lisible par un agent IA.
- **Assistance générale** (generalHelp) : traitement des questions générales (ex. politique de retour, conditions de livraison) sans requête SQL complexe.

4.4.5 Extraction et écriture des données (BDD) – Module Product Creation

Le workflow Product Creation lit et écrit explicitement dans la base de données :

- **Lecture catalogue** (Path B) : `SELECT name, unit_price, description FROM products ...` pour répondre aux requêtes conversationnelles produit. Colonnes exploitées : `products.name`, `products.unit_price`, `products.description`.
- **Lecture catégories** (Path A) : `SELECT id, name FROM categories;` pour choisir la meilleure catégorie via l'agent d'enrichissement. Colonnes exploitées : `categories.id`, `categories.name`.
- **Écriture produit** (Path A) : `INSERT INTO products (name, added_by, category_id, tags, description, unit_price) ....` Colonnes renseignées : `name`, `added_by`, `category_id`, `tags`, `description`, `unit_price`.

Le calcul métier ici est principalement un enrichissement algorithmique (prix manquant, catégorisation, tags, description) avant persistance SQL.

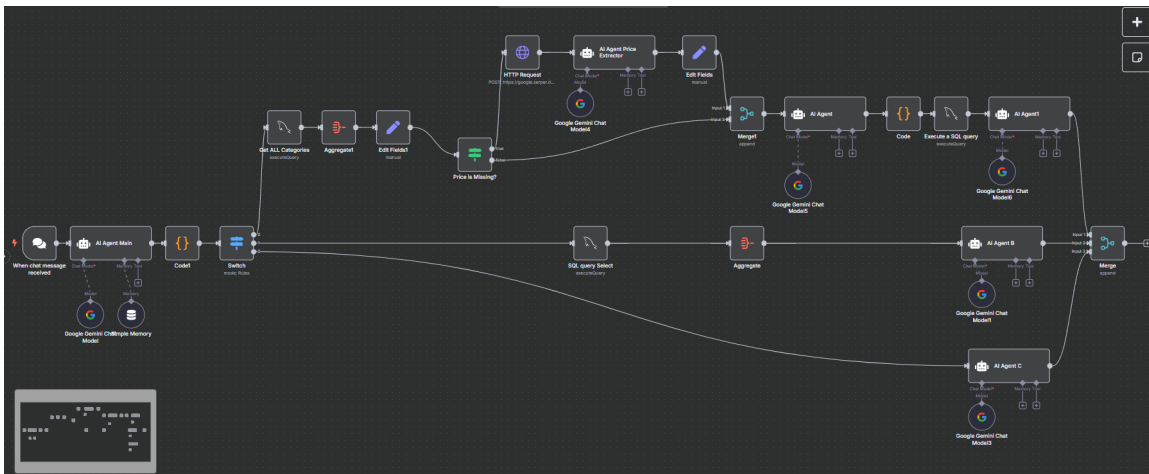


FIGURE 4.2 – Workflow de création assistée des fiches produits

Note de cohérence technique

Le fichier d'export local `Prigado Product Creation.json` contient bien le workflow Product Creation (28 nœuds). L'export confirme notamment :

- un trigger conversationnel (`When chat message received`),
- un routage par intention via `Switch` (`addProduct`, `queryProduct`, `generalHelp`),
- une branche prix manquant (`Price is Missing?` + `HTTP Request` + `AI Agent Price Extractor`),
- une branche requête produit (`SQL query Select` + `Aggregate`),
- une fusion finale des parcours via `Merge`.

4.4.6 Illustrations de la démonstration (présentation)

Les captures de démonstration mettent en évidence la valeur métier des trois parcours :

- **Path A** : ajout produit avec prix fourni et cas prix manquant (même capture).

- **Enrichissement** : génération automatique de tags produit.
- **Path B** : requête produit unitaire + liste complète (même capture).
- **Path C** : réponse FAQ (politique de livraison).

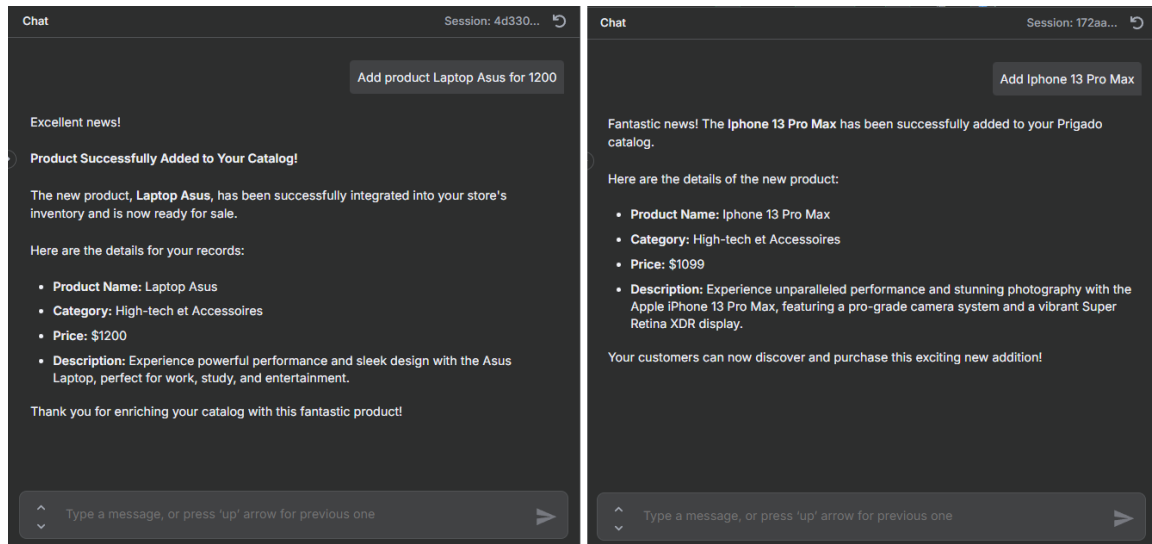


FIGURE 4.3 – Module 1 - Path A (prix fourni et prix manquant)

INPUT

Q

Schema

Table

JSON

Code

1 item

A

name

Iphone 13 Pro Max

#

category_id

137

A

category_name

High-tech et Accessoires

tags

A

tags[0]

iphone

A

tags[1]

smartphone

A

tags[2]

apple

A

description

Experience unparalleled performance and stunning photography with the Apple iPhone 13 Pro Max, featuring a pro-grade camera system and a vibrant Super Retina XDR display.

#

unit_price

1099

AI Agent

1 item

Merge1

1 item

Edit Fields

1 item

Price is Missing?

1 item

AI Agent Price Extractor

1 item

Edit Fields1

1 item

HTTP Request

1 item

Aggregate1

1 item

Get ALL Categories

35 items

FIGURE 4.4 – Module 1 - Génération de tags produit

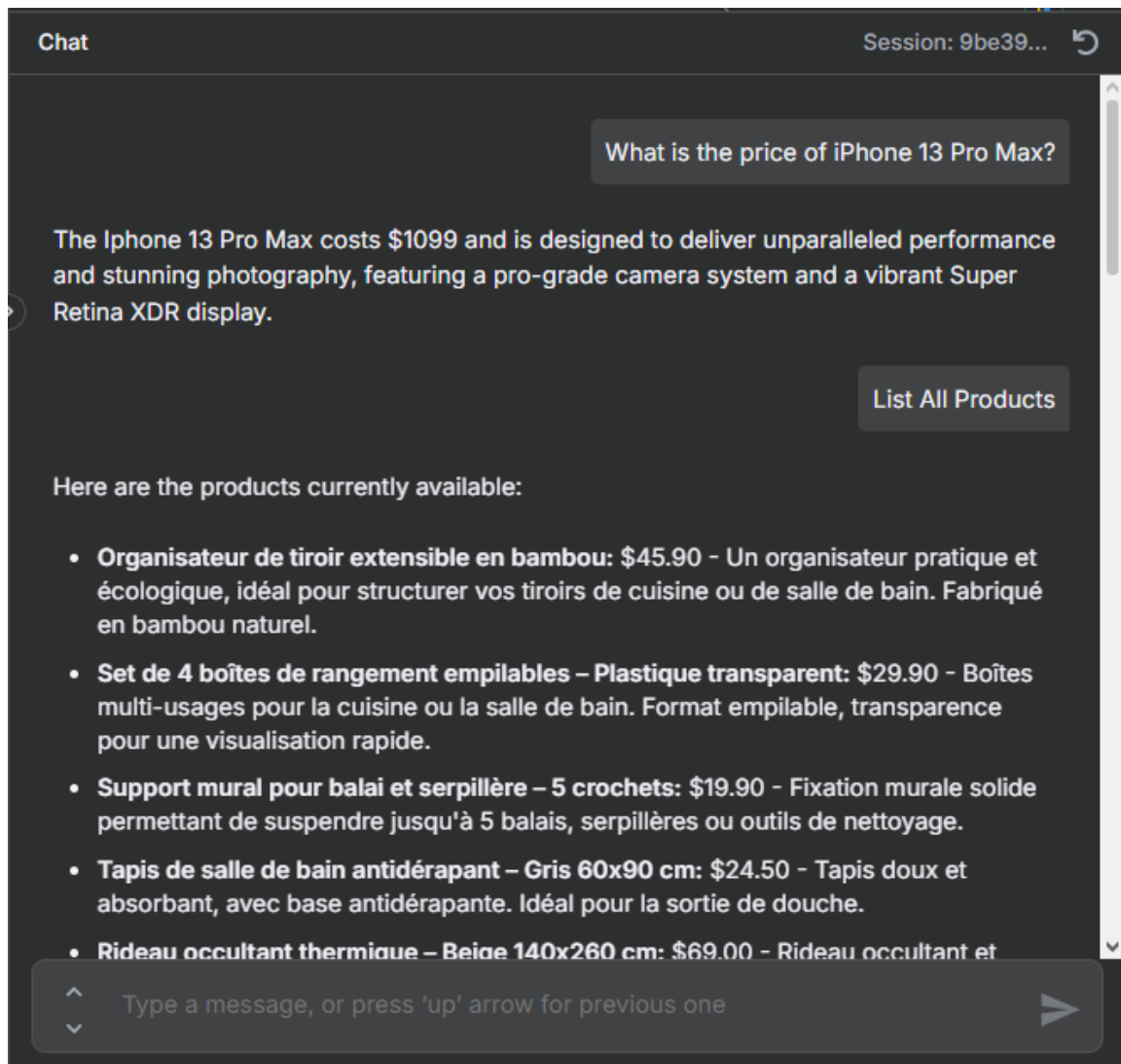


FIGURE 4.5 – Module 1 - Path B (produit unitaire et liste complète)

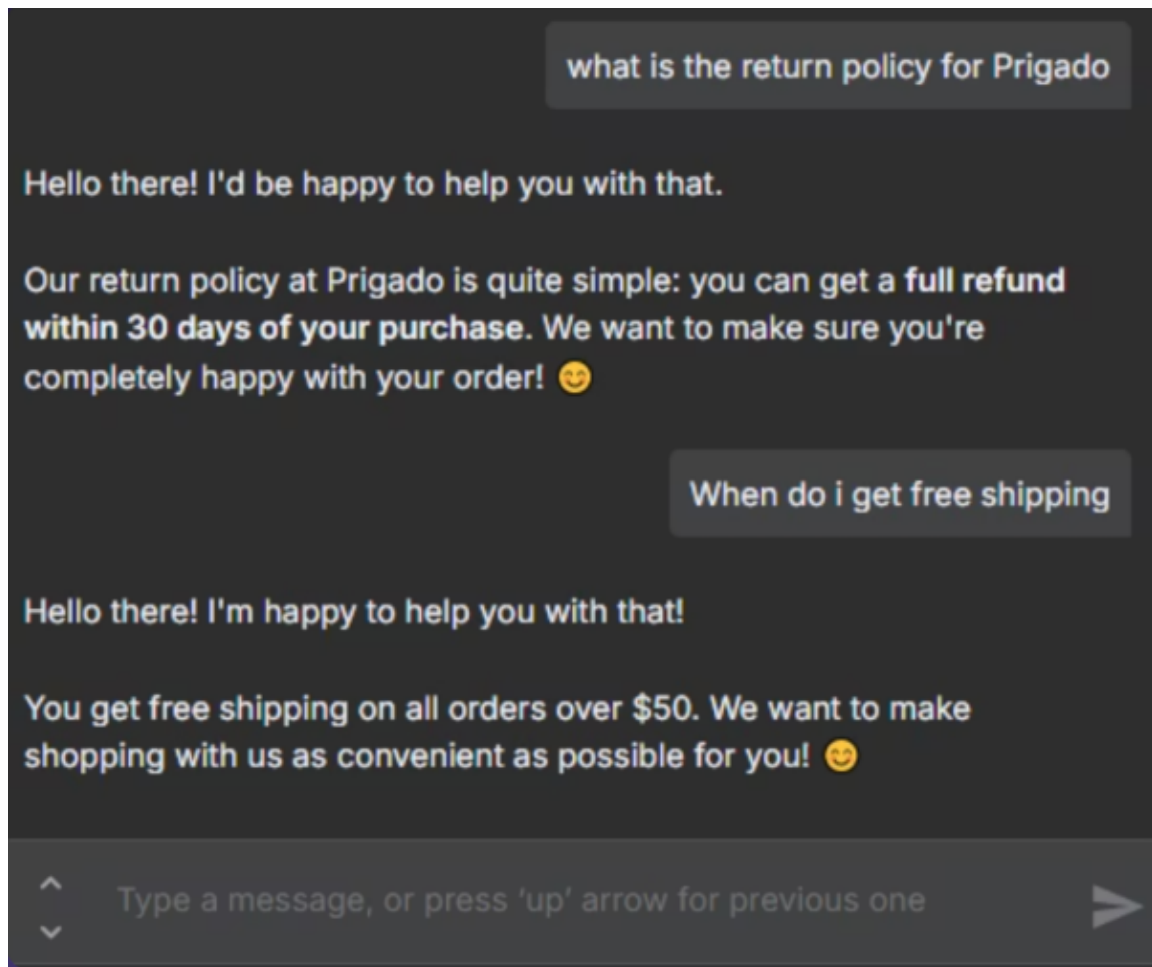


FIGURE 4.6 – Module 1 - Path C (generalHelp)

4.5 Module 2 – Prigado Boost : assistant ventes et marketing (Epic 5)

4.5.1 Intentions implémentées

Le workflow Prigado Boost implante quatre intentions principales :

- launchCampaign
- getSalesReport
- suggestPromo
- createSocialPost

4.5.2 Fonctions réalisées

1. **Campagnes email automatisées** : génération IA d'un email marketing (subject, body_text, body_html), personnalisation par client via Code Node, et envoi via

SMTP. Après l'envoi, les résultats sont agrégés et évalués par un agent IA pour informer l'utilisateur du nombre de mails envoyés avec succès.

2. **Rapport de ventes par vendeur** : résolution du vendeur, calcul SQL du chiffre d'affaires par produit, agrégation des résultats, reformulation IA lisible.
3. **Suggestion de bundle promotionnel** : extraction d'un large échantillon (jusqu'à 1000 lignes) des produits regroupés par `p.name` et `p.unit_price`, permettant au *AI Marketing Strategist* de sélectionner les 2 ou 3 articles les plus pertinents pour proposer un bundle.
4. **Création de contenu social** : normalisation des demandes vagues (ex. « top-selling products »), récupération conditionnelle des données (branche `If Topic is 'Top-Selling'?`), génération d'un JSON `post_text + suggested_image_idea` prêt à être consommé par une étape de publication (ex. Facebook Graph API).

4.5.3 Extraction et transfert des données (BDD) – Module Boost

Le module Boost s'appuie sur des lectures SQL ciblées, puis transfère les résultats vers des agents IA ou SMTP :

- **launchCampaign** : lecture des clients via `users(name, email, phone)` avec filtre `user_type='customer'`, puis génération/personnalisation du contenu avant envoi SMTP.
- **getSalesReport** : résolution du vendeur via `users.id`, puis calcul du chiffre d'affaires par produit avec `SUM(order_details.price * order_details.quantity)` en joignant `order_details.product_id = products.id`.
- **suggestPromo** : extraction d'un large échantillon (jusqu'à 1000 lignes) des produits regroupés par `p.name` et `p.unit_price`, permettant au *AI Marketing Strategist* de sélectionner les 2 ou 3 articles les plus pertinents.
- **createSocialPost** : récupération des top produits (lecture SQL), puis transformation en JSON social pré-publié.

Dans ce module, il n'y a pas d'écriture SQL de nouvelles lignes : le flux fait de l'extraction, du calcul/agrégation et de la restitution (texte marketing, rapport, email, JSON social).

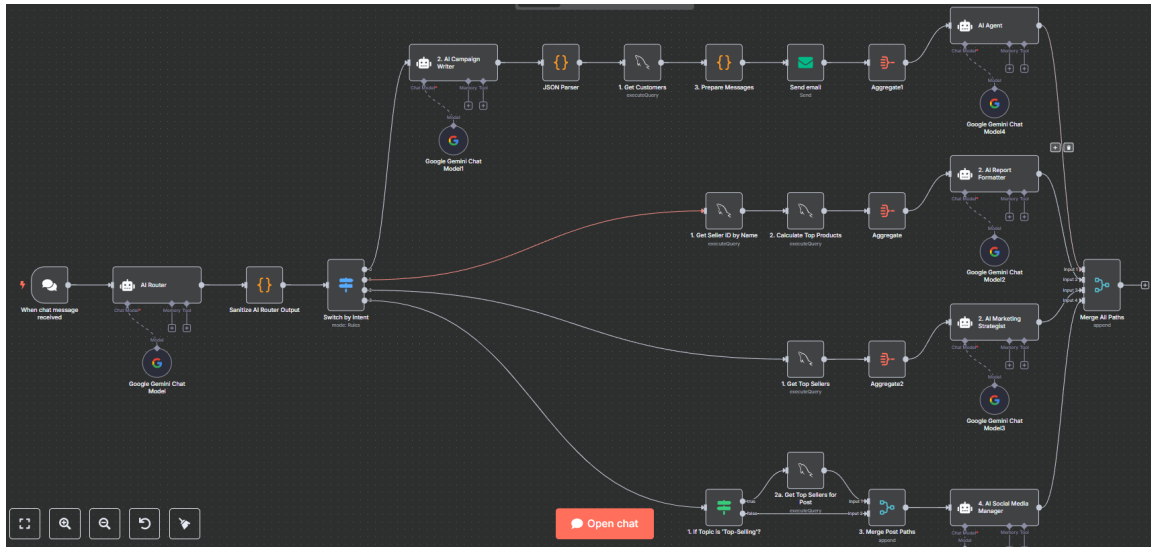


FIGURE 4.7 – Workflow Prigado Boost (ventes et marketing)

4.5.4 Illustrations de test du module Boost (chat + sorties)

Les captures suivantes documentent les tests de chaque path du module Boost :

- `launchCampaign` : message chat + email envoyé.
- `getSalesReport` : comparaison des rapports pour deux vendeurs.
- `suggestPromo` : proposition de bundle marketing.
- `createSocialPost` : post ciblé produit et idée d'image associée.
- `createSocialPost` (topic normalisé) : post sur tendances chaudes.

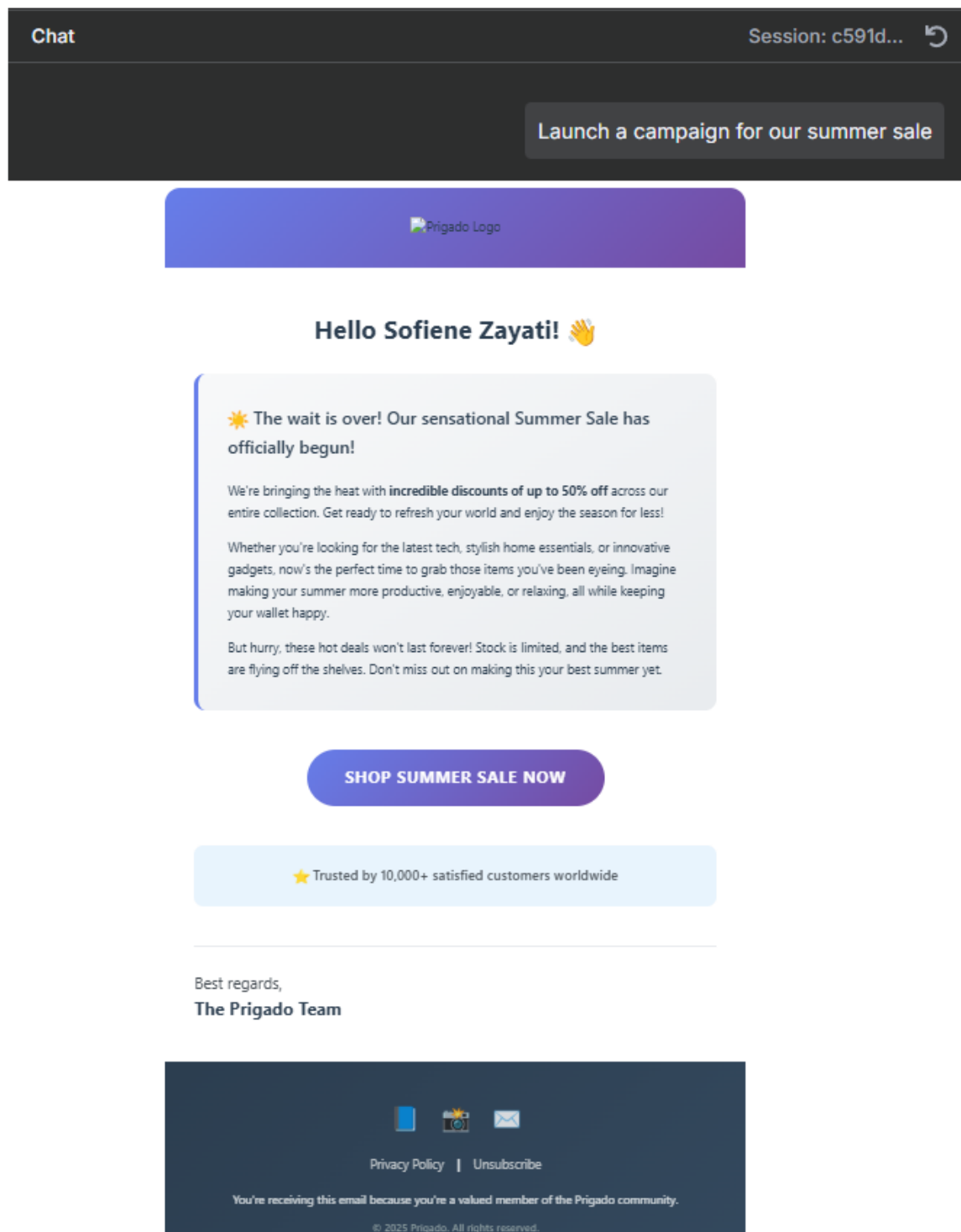


FIGURE 4.8 – Module 2 - Test launchCampaign (chat + email)

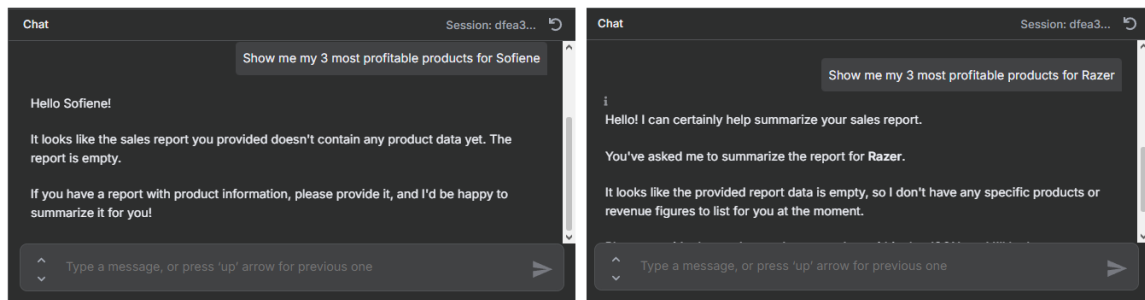


FIGURE 4.9 – Module 2 - Test getSalesReport (Sofiene et Razer)

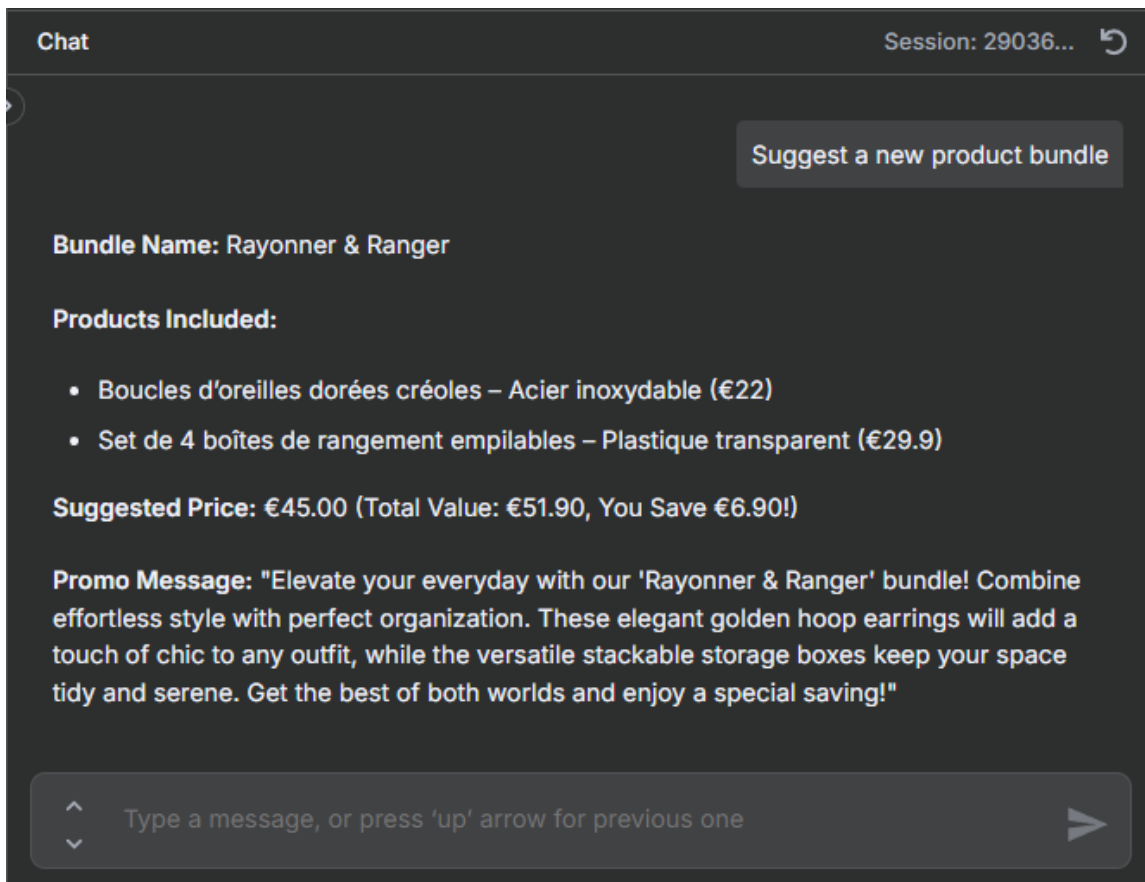


FIGURE 4.10 – Module 2 - Test suggestPromo

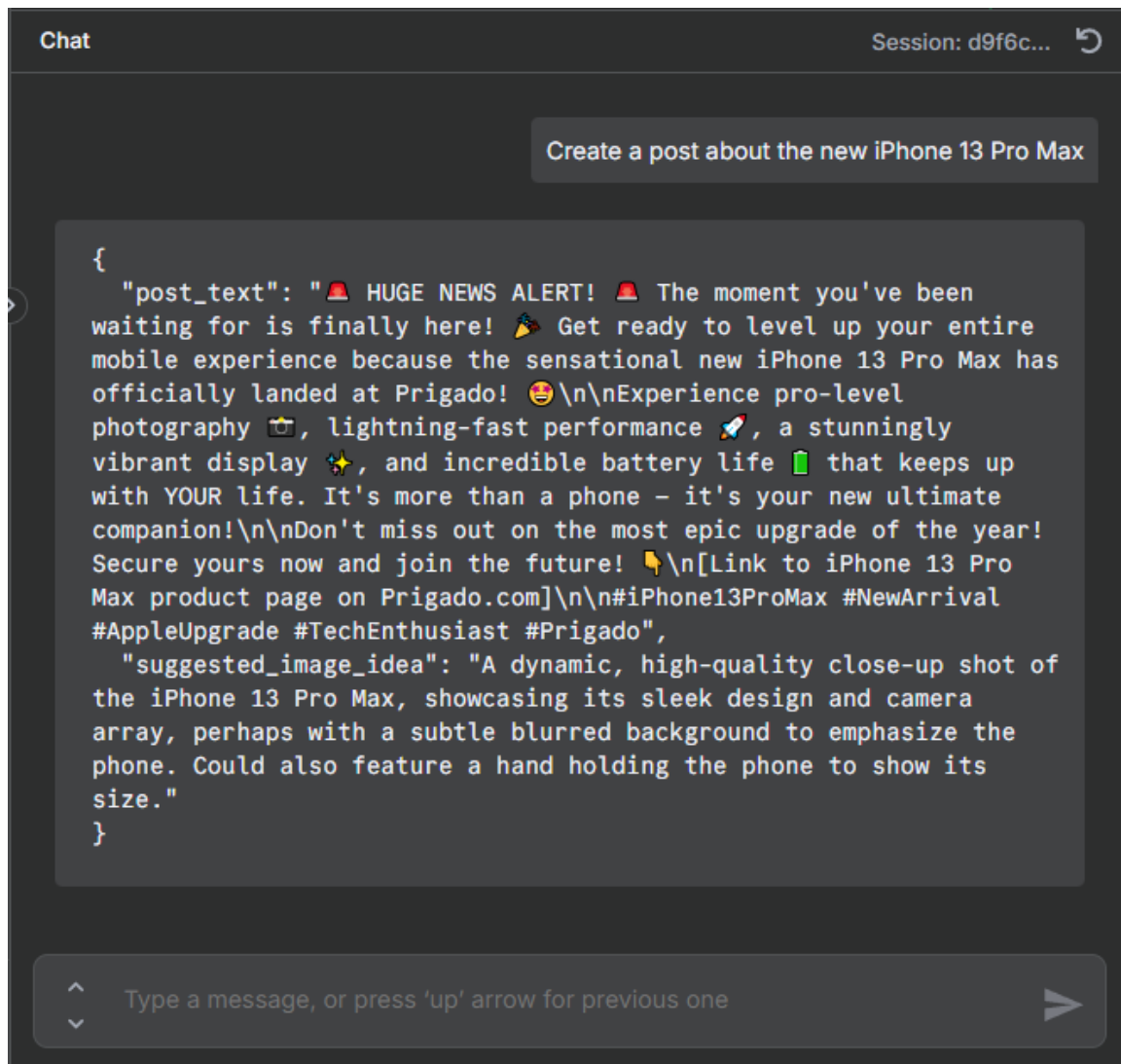


FIGURE 4.11 – Module 2 - Test createSocialPost (topic spécifique)

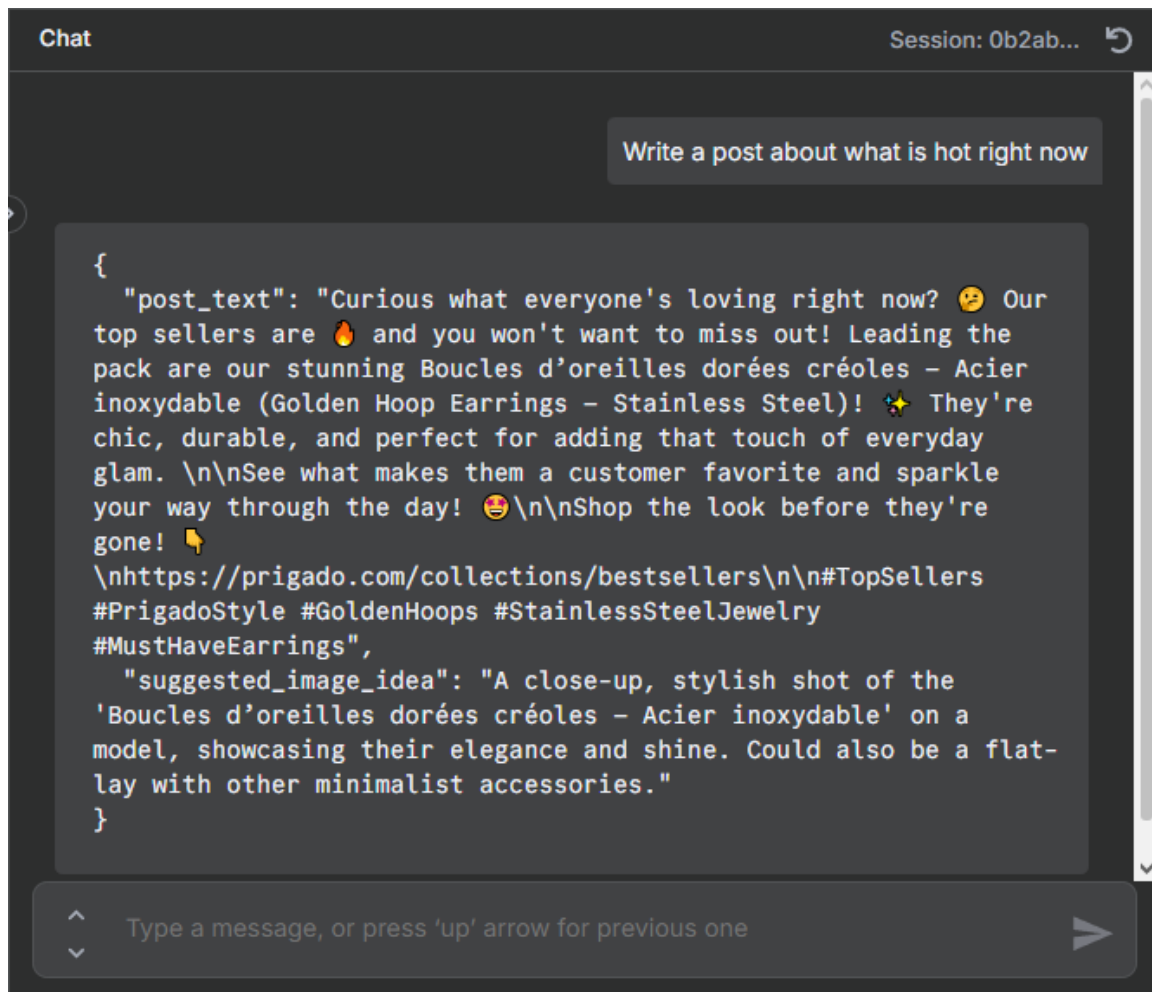


FIGURE 4.12 – Module 2 - Test createSocialPost (topic normalisé top-selling)

4.6 Module 3 – Prigado Logistics : recommandation logistique et commerciale (Epic 6)

4.6.1 Intentions implémentées

Le workflow Prigado Logistics implémente trois intentions :

- predictActivityPeak
- recommendTrendingProducts
- checkCustomerRisk

4.6.2 Fonctions réalisées

1. **Prédiction des pics d'activité** : analyse des commandes des 90 derniers jours et génération d'une alerte prédictive actionnable.

2. **Produits tendance** : extraction des top produits des 30 derniers jours et restitution sous forme de recommandations métier.
3. **Évaluation du risque client** : identification du client (nom/email), calcul des commandes totales vs annulées, puis classification du risque.

4.6.3 Extraction et algorithmes (BDD) – Module Logistics

Les trois intentions Logistics sont basées sur des agrégations SQL explicites :

- **predictActivityPeak** : extraction sur `order_details` avec fenêtre glissante 90 jours, `DATE(created_at)` et `COUNT(id)` AS `total_orders`, puis classement des jours par volume (`ORDER BY total_orders DESC LIMIT 10`). L'algorithme est un scoring de pic basé sur la fréquence journalière des commandes.
- **recommendTrendingProducts** : jointure `order_details` + `products` et calcul `SUM(quantity)` AS `total_sold` sur 30 jours pour classer les produits tendance.
- **checkCustomerRisk** : identification client via `users(name/email)`, puis calcul `total_orders = COUNT(orders.id)` et `cancelled_orders = SUM(CASE WHEN order_details THEN 1 END)` pour produire un ratio de risque et une classe Low/Medium/High.

Comme pour Boost, ce module est orienté lecture analytique : aucune insertion SQL n'est réalisée, les résultats sont calculés puis reformulés en sortie métier.

Le score de risque utilisé dans l'agent est fondé sur :

$$ScoreRisque = \begin{cases} 0 & \text{si } total_orders = 0 \\ \frac{cancelled_order}{total_orders} \times 100 & \text{sinon} \end{cases}$$

Classification appliquée dans l'agent IA :

- 0–10% : Low Risk
- 11–30% : Medium Risk
- 31% et plus : High Risk

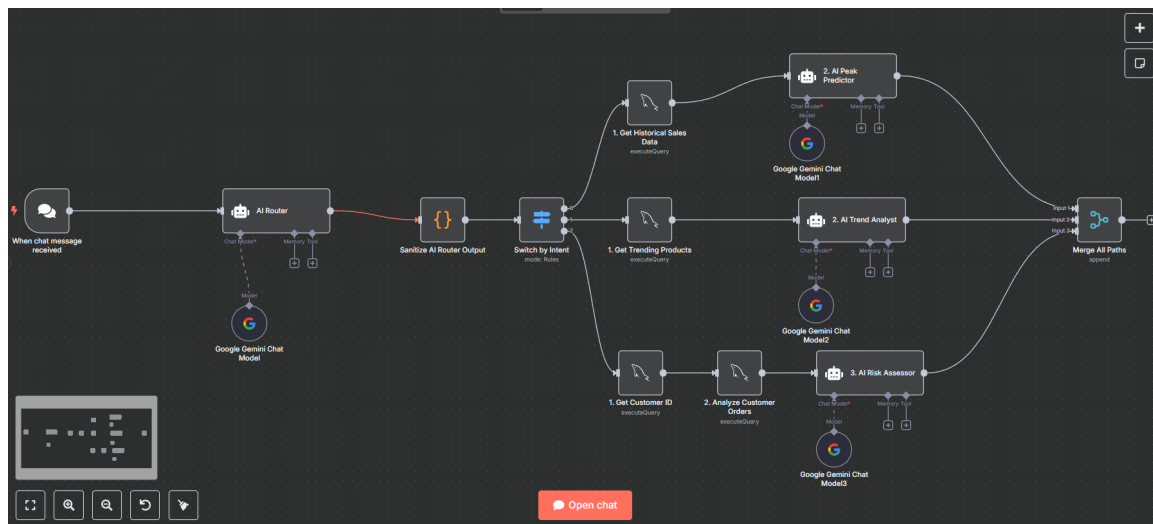


FIGURE 4.13 – Workflow Prigado Logistics (analyse prédictive et risque)

4.6.4 Illustrations de test du module Logistics (chat + sorties)

Compte tenu de l'inactivité temporaire de l'application, la capture des tendances peut afficher un retour sans données récentes (« No trending products were found in the last 30 days. »), ce qui est un comportement attendu du flux.

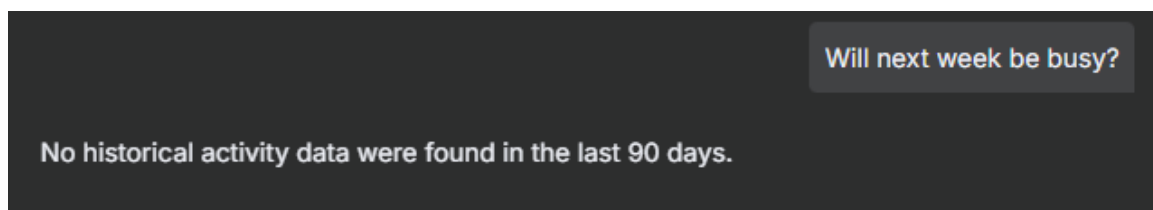


FIGURE 4.14 – Module 3 - Test predictActivityPeak

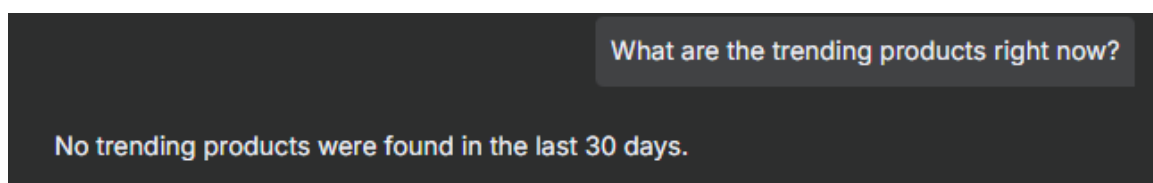


FIGURE 4.15 – Module 3 - Test recommendTrendingProducts (avec fallback no-data)

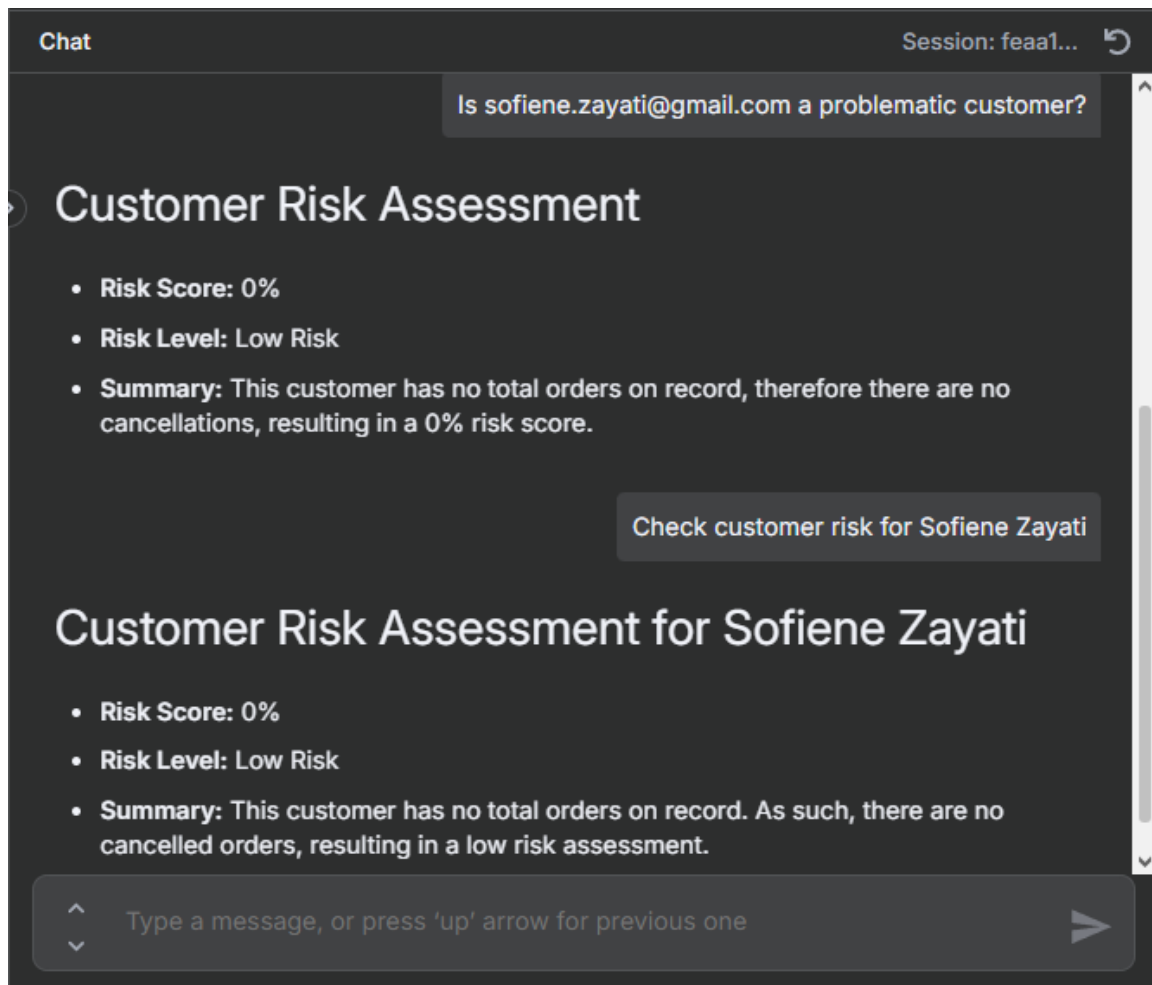


FIGURE 4.16 – Module 3 - Test checkCustomerRisk (email et nom client)

Chapitre 5

Défis techniques et solutions apportées

5.1 Défi 1 – Gestion de l’absence de données

5.1.1 Problème

Plusieurs branches pouvaient recevoir des résultats SQL vides. Sans gestion explicite, la sortie devenait ambiguë ou peu utile.

5.1.2 Solutions mises en œuvre

- **Approche IA (guard clauses)** : ajout de règles strictes dans les prompts pour imposer une réponse explicite si l’entrée est vide (ex. « No trending products were found in the last 30 days. »).
- **Approche logique (conditionnelle)** : utilisation du routage par intention, de branchements conditionnels (ex. `If Topic is 'Top-Selling'?`) et de `alwaysOutputData` sur certaines requêtes SQL pour garder une exécution robuste, même en cas de résultat vide. Sur les cas métier complexes, cette logique permet de distinguer des retours contextuels différents (ex. « vendeur non trouvé » vs « pas de données de vente »).

5.2 Défi 2 – Agrégation des résultats SQL pour l’IA

5.2.1 Problème

n8n traite les lignes SQL comme des items distincts, alors que les agents IA doivent souvent recevoir une vue consolidée.

5.2.2 Solution

Utilisation systématique de **Aggregate Node** pour transformer des sorties multiples en un tableau JSON unique avant appel des agents IA.

5.3 Défi 3 – Consistance des données entre branches

5.3.1 Problème

Dans certaines branches conditionnelles (notamment sur la création produit), les structures JSON différaient selon le chemin suivi.

5.3.2 Solution

Harmonisation des payloads via des nœuds de transformation (**Set / Code**) afin d'imposer un schéma de données unique avant fusion. Cette normalisation évite les conflits de mapping lors du **Merge Node** et stabilise les étapes en aval.

5.4 Défi 4 – Fiabilité des sorties IA

5.4.1 Problème

Les agents peuvent produire du texte non strictement JSON (texte additionnel, formatage markdown).

5.4.2 Solution

Insertion d'un nœud **Sanitize AI Router Output** (et parseurs JSON dédiés) basé sur des expressions régulières (spécifiquement le regex `\{[\s\S]*\}`) pour nettoyer les blocs Markdown générés par l'IA (comme les balises `"'json`) et fiabiliser l'exécution en aval.

Chapitre 6

Intégration avec la base de données

Les workflows s'appuient directement sur la base `db_ecommerce`.

6.1 Tables mobilisées

Table	Usage dans les workflows
<code>users</code>	Identification clients/vendeurs, ciblage email, résolution <code>seller_name</code> , score risque client.
<code>products</code>	Source produit pour analyses, suggestions promo et insertion produit assistée.
<code>orders</code>	Historique de commandes client (analyse de fiabilité).
<code>order_details</code>	Quantités, statuts de livraison, calcul CA et tendances.
<code>categories</code>	Référence de classification pour la création assistée de fiches produits.

6.2 Complexité relationnelle et choix de modélisation

L'un des points les plus techniques du projet a été la maîtrise des relations entre tables pour produire des réponses IA fiables. La difficulté ne venait pas seulement des requêtes SQL, mais du choix des bonnes clés de jointure et des bonnes colonnes métier selon le cas d'usage (vente, tendance, risque, création produit).

6.2.1 Relations exploitées dans les workflows

- `users.id` → `orders.user_id` : relier un client à ses commandes.
- `users.id` → `orders.seller_id` et `order_details.seller_id` : relier un vendeur à sa performance.

- `orders.id` → `order_details.order_id` : passer du niveau commande au niveau ligne produit.
- `products.id` → `order_details.product_id` : relier les ventes aux fiches produits.
- `categories.id` → `products.category_id` : relier produit et catégorie pour la création assistée.
- `categories.parent_id` → `categories.id` : gérer la hiérarchie parent/enfant des catégories.

6.2.2 Pourquoi la relation était complexe en pratique

- **Granularité différente** : `orders` contient une vue globale de commande, alors que `order_details` contient la vérité au niveau ligne (`quantity`, `delivery_status`, `created_at`, `seller_id`). Pour les analyses fines, il fallait privilégier `order_details`.
- **Colonnes proches mais sémantiquement différentes** : la date `orders.date` (timestamp numérique) coexiste avec `order_details.created_at` (timestamp SQL). Les fenêtres 30/90 jours des workflows ont été calculées sur `order_details.created_at` pour rester cohérentes avec les jointures produit.
- **Identification robuste des acteurs** : pour retrouver un vendeur ou un client depuis une requête naturelle, il fallait combiner `name/email` avec `user_type` afin d'éviter les ambiguïtés entre profils (`admin`, `seller`, `customer`, `delivery_boy`).
- **Indicateurs métier dérivés** : le score de risque n'existe pas en base. Il est reconstruit à la volée en combinant `COUNT(...)` et `SUM(CASE WHEN ...)` sur plusieurs tables.

6.2.3 Colonnes clés mobilisées pour obtenir les meilleurs résultats

- `users` : `id`, `user_type`, `name`, `email`.
- `orders` : `id`, `user_id`, `seller_id`, `delivery_status`, `date`, `grand_total`.
- `order_details` : `order_id`, `seller_id`, `product_id`, `quantity`, `price`, `delivery_status`, `created_at`.
- `products` : `id`, `name`, `category_id`, `unit_price`.
- `categories` : `id`, `parent_id`, `level`, `name`.

6.2.4 Impact direct sur la qualité des sorties IA

Ce travail relationnel a permis d'améliorer nettement la pertinence des sorties :

- rapports vendeurs basés sur un chiffre d'affaires calculé correctement (`price` × `quantity`),
- tendances produits limitées à une fenêtre temporelle fiable,

- classification risque client reproductible et explicable,
- insertion produit enrichie avec une catégorisation cohérente.

6.3 Exemple de requêtes SQL utilisées

Listing 6.1 – Résolution du vendeur par nom (workflow Boost)

```
SELECT id
FROM users
WHERE name LIKE CONCAT('%', {{ seller_name }}, '%')
      AND user_type = 'seller'
LIMIT 1;
```

Listing 6.2 – Top produits d'un vendeur par chiffre d'affaires

```
SELECT
    p.name,
    SUM(od.price * od.quantity) AS total_revenue
FROM order_details od
JOIN products p ON od.product_id = p.id
WHERE od.seller_id = {{ seller_id }}
GROUP BY p.name
ORDER BY total_revenue DESC
LIMIT {{ limit }};
```

Listing 6.3 – Détection des pics d'activité sur 90 jours (workflow Logistics)

```
SELECT
    DATE(created_at) AS order_date,
    COUNT(id) AS total_orders
FROM order_details
WHERE created_at >= NOW() - INTERVAL 90 DAY
GROUP BY order_date
ORDER BY total_orders DESC
LIMIT 10;
```

Listing 6.4 – Tendances produits sur 30 jours

```
SELECT
    p.name,
    SUM(od.quantity) AS total_sold
FROM order_details od
JOIN products p ON od.product_id = p.id
WHERE od.created_at >= NOW() - INTERVAL 30 DAY
```

```
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 5;
```

Listing 6.5 – Résolution de l’identifiant client (workflow Logistics)

```
SELECT id
FROM users
WHERE email = '{{customer_identifier}}'
      OR name = '{{customer_identifier}}'
LIMIT 1;
```

Listing 6.6 – Analyse des commandes client pour le score de risque (workflow Logistics)

```
SELECT
  COUNT(o.id) AS total_orders,
  SUM(CASE WHEN od.delivery_status = 'cancelled' THEN 1 ELSE 0 END) AS
    cancelled_orders
FROM orders o
JOIN order_details od ON o.id = od.order_id
WHERE o.user_id = {{ customer_id }};
```

Listing 6.7 – Insertion d’un produit enrichi (workflow Product Creation)

```
INSERT INTO products (name, added_by, category_id, tags, description,
  unit_price)
VALUES (
  '{{name}}',
  'AI',
  {{ category_id }},
  '{{tags_json}}',
  '{{description}}',
  {{ unit_price }}
);
```

Conclusion générale et perspectives

L'intégration des assistants IA au sein de la plateforme Prigado a permis de démontrer qu'il est possible de transformer des processus métier traditionnels en automatisations intelligentes. Ce projet a généré un impact direct sur l'efficacité opérationnelle : automatisation de tâches manuelles chronophages (comme la rédaction de fiches produits au catalogue), gain de temps significatif pour les équipes, et structuration de la donnée métier via des flux orchestrés par n8n.

Limites du système

Malgré les bénéfices apportés, cette architecture présente certaines limitations qu'il convient de prendre en compte :

- **Dépendance aux API externes** : le fonctionnement repose fortement sur la disponibilité de Google Gemini et Serper.dev.
- **Latence des workflows n8n** : les multiples manipulations de JSON, Switchs et appels réseau IA introduisent inévitablement un temps de traitement pour l'utilisateur.
- **Qualité dépendante des données SQL** : la pertinence des réponses métier reste strictement corrélée à la qualité et la propreté de la base de données.

Perspectives et améliorations futures

Au-delà des usages actuels, le système gagnerait à évoluer sur le plan technique et de l'infrastructure :

- **Scalabilité** : déploiement des agents sur des infrastructures cloud à haute disponibilité pour absorber la charge lors des pics e-commerce.
- **Monitoring et observabilité** : mise en place de tableaux de bord pour surveiller les coûts d'API, les taux d'erreur et le temps d'exécution des workflows.
- **Optimisation des coûts** : évaluation de modèles open-source hébergés localement pour réduire la dépendance financière aux API propriétaires.

Bilan du stage

Ce stage m'a permis de mener un projet d'ingénierie de bout en bout : analyse du besoin, conception d'architecture, implémentation n8n, intégration IA (Google Gemini), connexion aux données MySQL, et production de solutions métier directement exploitables.

Au-delà des aspects techniques, cette expérience a renforcé ma capacité à traduire des besoins business complexes en workflows fiables, à anticiper les cas d'erreur avec des solutions de *fallback*, et à industrialiser l'usage de l'intelligence artificielle. Le projet Prigado a prouvé qu'une architecture IA bien structurée peut accélérer la création de valeur opérationnelle (marketing, vente, logistique et support décisionnel).

Bibliographie

1. Documentation n8n, <https://docs.n8n.io/>
2. Documentation Google Gemini API, <https://ai.google.dev/>
3. Documentation MySQL 8.0, <https://dev.mysql.com/doc/>
4. Documentation SMTP (IETF RFC 5321)
5. Documentation Serper.dev, <https://serper.dev/>